

Autonomous Puzzlebot Navigation

EKF Localization with ArUco Markers & A* Path Planning

Alfonso Diaz

Manchester Robotics × NVIDIA
Tecnológico de Monterrey · TE3003B

Final Challenge · Week 7

From scratch with NumPy + Python stdlib only

ROS 2 Humble · Gazebo Harmonic

Mission

Part 1 — EKF Localization

Estimate the robot pose (x, y, θ) from wheel encoders and ArUco marker observations.

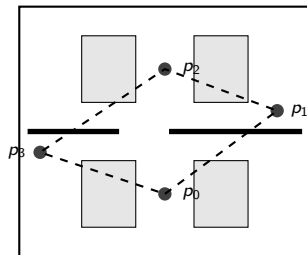
Part 2 — Multi-Waypoint Navigation

Drive the Puzzlebot through a closed trajectory of ≥ 4 waypoints inside a maze, avoiding walls.

Constraint

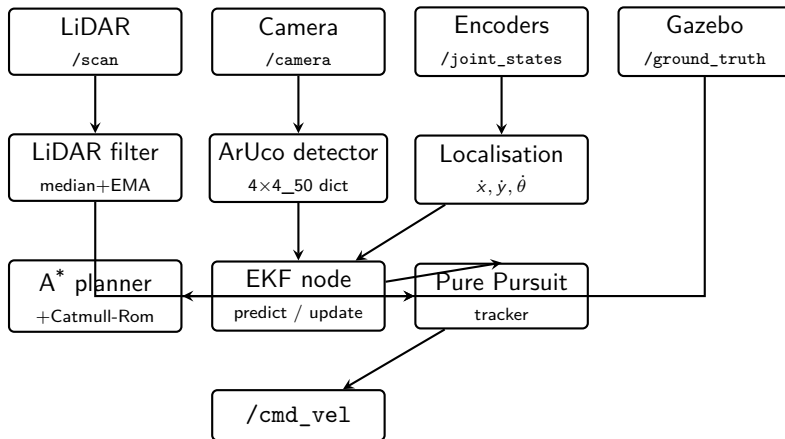
Only `numpy` + `Python` `stdlib`.

No `scipy`, no `Nav2`, no `SLAM` toolbox. Everything from scratch.



Maze with 4 inner chambers
and closed waypoint loop

System architecture



Every block: pure NumPy + ROS 2 Python. No external planners.

Differential drive kinematics

Wheel velocities ω_r, ω_l are read from encoders. Body twist:

$$v = \frac{r}{2}(\omega_r + \omega_l), \quad \omega = \frac{r}{L}(\omega_r - \omega_l)$$

where $r = 0.05$ m (wheel radius) and $L = 0.19$ m (wheelbase).

Discrete pose propagation with sample time Δt :

$$\begin{bmatrix} x_{k+1} \\ y_{k+1} \\ \theta_{k+1} \end{bmatrix} = \begin{bmatrix} x_k \\ y_k \\ \theta_k \end{bmatrix} + \Delta t \begin{bmatrix} v_k \cos \theta_k \\ v_k \sin \theta_k \\ \omega_k \end{bmatrix}$$

This is the *nominal* motion model — it drifts because of slip, quantization and unmodelled dynamics. We need observations to correct it.

EKF — Prediction step

State $\boldsymbol{\mu}_k = [x_k, y_k, \theta_k]^\top$, covariance $\Sigma_k \in \mathbb{R}^{3 \times 3}$.

Mean propagation (nominal kinematics, $f(\boldsymbol{\mu}_{k-1}, u_k)$):

$$\boldsymbol{\mu}_{k|k-1} = f(\boldsymbol{\mu}_{k-1}, u_k)$$

Covariance propagation with Jacobian $F_k = \partial f / \partial \boldsymbol{\mu}$:

$$\Sigma_{k|k-1} = F_k \Sigma_{k-1} F_k^\top + Q_k$$

$$F_k = \begin{bmatrix} 1 & 0 & -\Delta t v_k \sin \theta_{k-1} \\ 0 & 1 & \Delta t v_k \cos \theta_{k-1} \\ 0 & 0 & 1 \end{bmatrix}$$

Process noise Q_k derives from per-wheel proportional noise:

$$Q_k = B_k \text{diag}(k_r |\omega_r|, k_l |\omega_l|) B_k^\top, \quad k_r = k_l = 0.15$$

EKF — Observation model (range-bearing)

For ArUco marker i at known map location (m_x^i, m_y^i) , the camera measures:

$$z = \begin{bmatrix} r \\ \beta \end{bmatrix} = \begin{bmatrix} \sqrt{(m_x^i - x)^2 + (m_y^i - y)^2} \\ \arctan 2(m_y^i - y, m_x^i - x) - \theta \end{bmatrix}$$

Measurement Jacobian $H_k = \partial h / \partial \mu$:

$$H_k = \begin{bmatrix} -\frac{\Delta x}{r} & -\frac{\Delta y}{r} & 0 \\ \frac{\Delta y}{r^2} & -\frac{\Delta x}{r^2} & -1 \end{bmatrix}$$

with $\Delta x = m_x^i - x$, $\Delta y = m_y^i - y$.

Observation noise: $R = \text{diag}(\sigma_r^2, \sigma_\beta^2)$ with $\sigma_r = 0.08 \text{ m}$, $\sigma_\beta = 0.06 \text{ rad } (\approx 3.5^\circ)$.

EKF — Update step & outlier rejection

Innovation ν_k and its covariance S_k :

$$\nu_k = z_k - h(\mu_{k|k-1}), \quad S_k = H_k \Sigma_{k|k-1} H_k^\top + R$$

Wrap the bearing component of ν_k into $(-\pi, \pi]$ before using it.

Mahalanobis gate — reject observations with

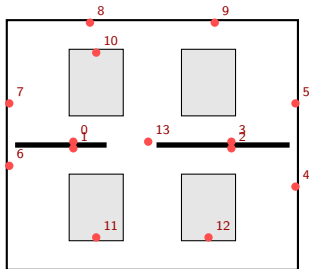
$$d^2 = \nu_k^\top S_k^{-1} \nu_k > \chi_{2, 0.999}^2 \approx 30$$

protects the filter against false detections / wrong marker IDs.

Kalman gain & update:

$$K_k = \Sigma_{k|k-1} H_k^\top S_k^{-1}$$
$$\mu_k = \mu_{k|k-1} + K_k \nu_k, \quad \Sigma_k = (I - K_k H_k) \Sigma_{k|k-1}$$

ArUco map (14 markers)



14 markers covering every corridor segment. From any waypoint, the robot sees ≥ 2 markers within 2.5 m — guaranteed redundancy.

Covariance ellipse (visual interpretation)

Project the 2×2 XY block of Σ_k onto the plane:

$$\Sigma_{xy} = \begin{bmatrix} \sigma_{xx} & \sigma_{xy} \\ \sigma_{xy} & \sigma_{yy} \end{bmatrix}, \quad \Sigma_{xy} = V \Lambda V^T$$

with eigenvalues $\Lambda = \text{diag}(\lambda_1, \lambda_2)$ and eigenvectors V .

95% confidence ellipse (2 dof, $\chi^2 = 5.991$):

$$a = \sqrt{\chi^2 \lambda_{\max}}, \quad b = \sqrt{\chi^2 \lambda_{\min}}$$

Rotation angle: $\alpha = \arctan 2(v_{\max,y}, v_{\max,x})$.

In RViz: the ellipse **grows** on stretches without markers (uncertainty accumulates) and **shrinks** when an ArUco update lands — visual proof the filter is working.

Occupancy grid & A* planner

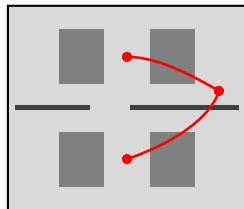
Grid build from YAML: each wall is a rectangle in metric coords. Discretized to 140×120 cells (resolution 5 cm).

Configuration-space inflation: dilate occupied cells by $r = 0.28$ m so the planner never produces paths that clip a wall.

A* on 8-connected grid:

$$f(n) = g(n) + h(n), \quad h(n) = \sqrt{(x_n - x_g)^2 + (y_n - y_g)^2}$$

Implemented with a binary heap (heapq) —
admissible heuristic guarantees optimality.



Inflated grid + A* solution

Catmull-Rom path smoothing

A^* returns a polyline with sharp corners. We smooth it with a **Catmull-Rom spline** for C^1 continuity.

For 4 consecutive control points $P_{i-1}, P_i, P_{i+1}, P_{i+2}$, the segment between P_i and P_{i+1} is parameterized by $t \in [0, 1]$:

$$P(t) = \frac{1}{2} \begin{bmatrix} 1 & t & t^2 & t^3 \end{bmatrix} \begin{bmatrix} 0 & 2 & 0 & 0 \\ -1 & 0 & 1 & 0 \\ 2 & -5 & 4 & -1 \\ -1 & 3 & -3 & 1 \end{bmatrix} \begin{bmatrix} P_{i-1} \\ P_i \\ P_{i+1} \\ P_{i+2} \end{bmatrix}$$

Properties used:

- Passes through every P_i (interpolating, not approximating).
- Tangent at P_i is parallel to $P_{i+1} - P_{i-1}$.
- Local: changing one control point only affects the two adjacent segments.

We sample 25 points per segment $\Rightarrow \approx 3$ cm between consecutive samples.

Pure Pursuit tracker

Pick a lookahead point (x_L, y_L) on the smoothed path at distance $L_d = 0.30$ m from the robot.

Express it in the body frame:

$$\begin{bmatrix} x_L^b \\ y_L^b \end{bmatrix} = R(-\theta) \begin{bmatrix} x_L - x \\ y_L - y \end{bmatrix}$$

Curvature of the arc that reaches the lookahead point:

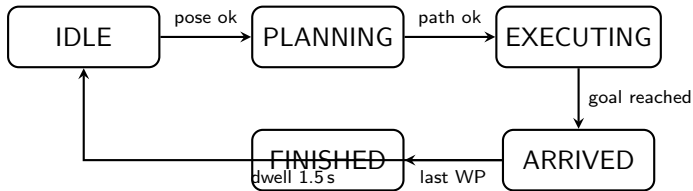
$$\kappa = \frac{2 y_L^b}{L_d^2}$$

Output velocities:

$$v = v_{\max} = 0.18 \text{ m/s}, \quad \omega = v \kappa$$

Near the goal, v is ramped down linearly inside the brake distance (0.30 m). LiDAR safety layer intercepts if frontal range < 0.15 m.

Finite State Machine



IDLE → wait for first pose. **PLANNING** → run A^* + smooth + arm tracker.

EXECUTING → Pure Pursuit publishes `cmd_vel` at 30 Hz.

ARRIVED → stop, dwell, advance to next waypoint.

FINISHED → closed loop done, brake & hold.

Part 1 — EKF metrics

Localization accuracy

Trajectory averaged over the maze loop,
against /ground_truth:

Metric	Value
RMSE position	≈ 7 mm
Detection rate	99%
NEES in band	87.5%
Mahalanobis rejects	< 5%

NEES band: $\chi^2_{2,0.025} \leq \text{NEES} \leq \chi^2_{2,0.975}$
 $\Rightarrow$ filter is statistically consistent.

What the demo shows

- Covariance ellipse *breathes* as markers come into view.
- No marker visible $\Rightarrow$ ellipse **grows**.
- ArUco update lands $\Rightarrow$ ellipse **shrinks**.
- Trail of past ellipses traces the uncertainty history along the path.

Part 2 — Navigation result

Closed trajectory completed:

$$p_0 \rightarrow p_1 \rightarrow p_2 \rightarrow p_3 \rightarrow p_0$$

Heading change at every leg $> 50^\circ$ (PDF requirement: no two consecutive co-directional legs). The robot circumnavigates all four inner chambers.

No collisions. The configuration-space inflation ($r = 0.28$ m) guarantees every A^* path stays ≥ 28 cm from any wall, even with EKF drift.

Pipeline runs from scratch: A^* , Catmull-Rom, Pure Pursuit, EKF — all implemented with NumPy only.

Property	Value
Lap time	≈ 75 s
Max speed $v_{\max}$	0.18 m/s
Max angular $\omega_{\max}$	0.8 rad/s
Lookahead L_d	0.30 m
Grid resolution	0.05 m
Robot radius (C-space)	0.28 m
Control rate	30 Hz
Planning rate	1 Hz

Why this satisfies the PDF rubric

- **Part 1:** EKF with prediction, ArUco-based update, confidence ellipses, three PDF scenarios (multi / no / partial marker) — ✓
- **Part 2:** 4 waypoints, closed trajectory, no straight legs, no co-directional consecutive legs, circumnavigates obstacles — ✓
- **Optional A:** global planner with A^* + spline smoothing instead of pure reactive — ✓
- **Optional B:** EKF runs in parallel during navigation and is plotted in RViz with the covariance ellipse — ✓
- **Library constraint:** only NumPy + Python stdlib, no scipy, no Nav2, no SLAM toolbox — ✓

Live demo plan

Timeline of the recorded video:

1. **0–10 s** — overview: RViz + Gazebo side by side.
2. **10–30 s** — Part 1 in action: zoom on the covariance ellipse, show how it shrinks when an ArUco lands and grows on long stretches.
3. **30–75 s** — Part 2 timelapse ($\times 4$): robot completes the full $p_0 \rightarrow p_1 \rightarrow p_2 \rightarrow p_3 \rightarrow p_0$ loop, planner path visible in green, LiDAR sweep visible in red.
4. **75–90 s** — final frame: *Mission complete*, robot stops at the closing waypoint.

Reproduce with:

```
./run.sh part2-astar
```

Thank you

Alfonso Diaz

personaldiaz01@gmail.com

Code: challenges/Week7/Challenge/
Run with ./run.sh part2-astar

MCR2 × NVIDIA · TE3003B · 2026